

Data and Artificial Intelligence

Cyber Shujaa Program

Week 11 Assignment – Natural Language Processing

(BERT Sentence Similarity)

Student Name: Hope Njoki Nyambura

Student ID: cs-eh03-25100

Table of Contents

1. Introduction	2
2. Data Description	2
3. Methods and Model Description	2
3.1 BERT Tokenizer and Preprocessing.....	2
3.2 BERT Model (Encoder)	3
3.3 Sentence Embedding Extraction	4
3.4 Cosine Similarity and Prediction	4
4. Results	5
4.1 Similarity Scores and Predictions.....	5
4.2 Accuracy Calculation	6
5. Key NLP Concepts.....	6
5.1 Difference Between BERT and Traditional NLP Models.....	6
5.2 Role of the BERT Encoder	7
5.3 Contextual Embeddings	7
5.4 Importance of the [CLS] Token	7
5.5 Cosine Similarity.....	7
6. Conclusion.....	8
7. Google Colab Link.....	8

1. Introduction

The purpose of this assignment was to explore Natural Language Processing (NLP) using a transformer-based model, specifically BERT (Bidirectional Encoder Representations from Transformers). The main objective was to evaluate the semantic similarity between pairs of sentences by generating contextual embeddings and comparing them using cosine similarity.

Ten sentence pairs were analyzed five provided in the assignment instructions and five created manually. Each pair was assigned a ground truth label indicating whether the sentences were semantically similar (1) or not similar (0). Using BERT embeddings and a similarity threshold of 0.7, the goal was to predict similarity and compute the model's accuracy against the manually assigned labels.

This report explains the methods used, presents the results obtained, and discusses key NLP concepts related to BERT and cosine similarity.

2. Data Description

The dataset consisted of 10 sentence pairs, each containing two English sentences. These included question-based pairs, descriptive sentences, and unrelated statements. The diversity of the pairs allowed for testing BERT's ability to understand context, meaning, and semantic relationships.

For each pair:

- A BERT embedding was generated for each sentence.
- Cosine similarity was computed between the two embeddings.
- A threshold of 0.7 determined whether the pair was classified as similar or not.

Ground truth labels were manually assigned based on human judgment of semantic similarity.

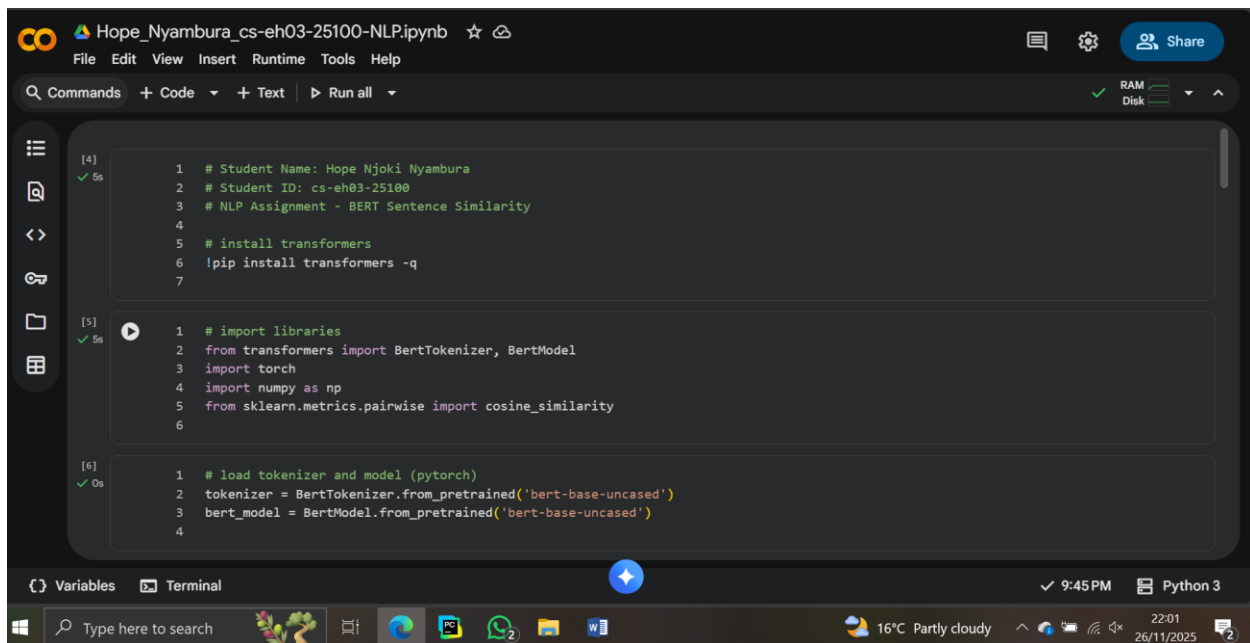
3. Methods and Model Description

3.1 BERT Tokenizer and Preprocessing

The *bert-base-uncased* tokenizer was used to:

- Convert each sentence into tokens
- Add required special tokens ([CLS], [SEP])
- Convert tokens into numerical IDs
- Prepare inputs for the BERT model

Padding and truncation ensured consistent input shapes.



```
[4] ✓ 5s
1 # Student Name: Hope Njoki Nyambura
2 # Student ID: cs-eh03-25100
3 # NLP Assignment - BERT Sentence Similarity
4
5 # install transformers
6 !pip install transformers -q
7

[5] ✓ 5s
1 # import libraries
2 from transformers import BertTokenizer, BertModel
3 import torch
4 import numpy as np
5 from sklearn.metrics.pairwise import cosine_similarity
6

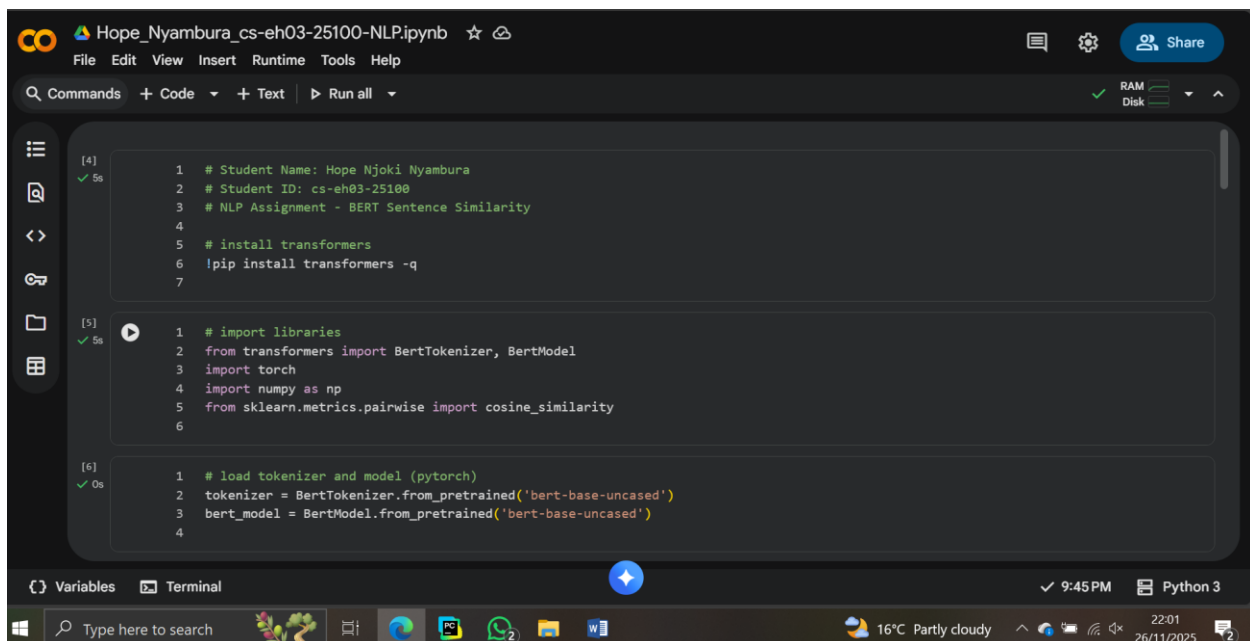
[6] ✓ 0s
1 # load tokenizer and model (pytorch)
2 tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
3 bert_model = BertModel.from_pretrained('bert-base-uncased')
4
```

3.2 BERT Model (Encoder)

The *bert-base-uncased* model was used for generating contextual embeddings. This model:

- Contains 12 encoder layers
- Produces 768-dimensional token embeddings
- Utilizes self-attention to understand relationships between words

The [CLS] token embedding was extracted as the sentence representation because BERT is trained to consolidate full-sequence meaning into this position.



```
[4] ✓ 5s
1 # Student Name: Hope Njoki Nyambura
2 # Student ID: cs-eh03-25100
3 # NLP Assignment - BERT Sentence Similarity
4
5 # install transformers
6 !pip install transformers -q
7

[5] ✓ 5s
1 # import libraries
2 from transformers import BertTokenizer, BertModel
3 import torch
4 import numpy as np
5 from sklearn.metrics.pairwise import cosine_similarity
6

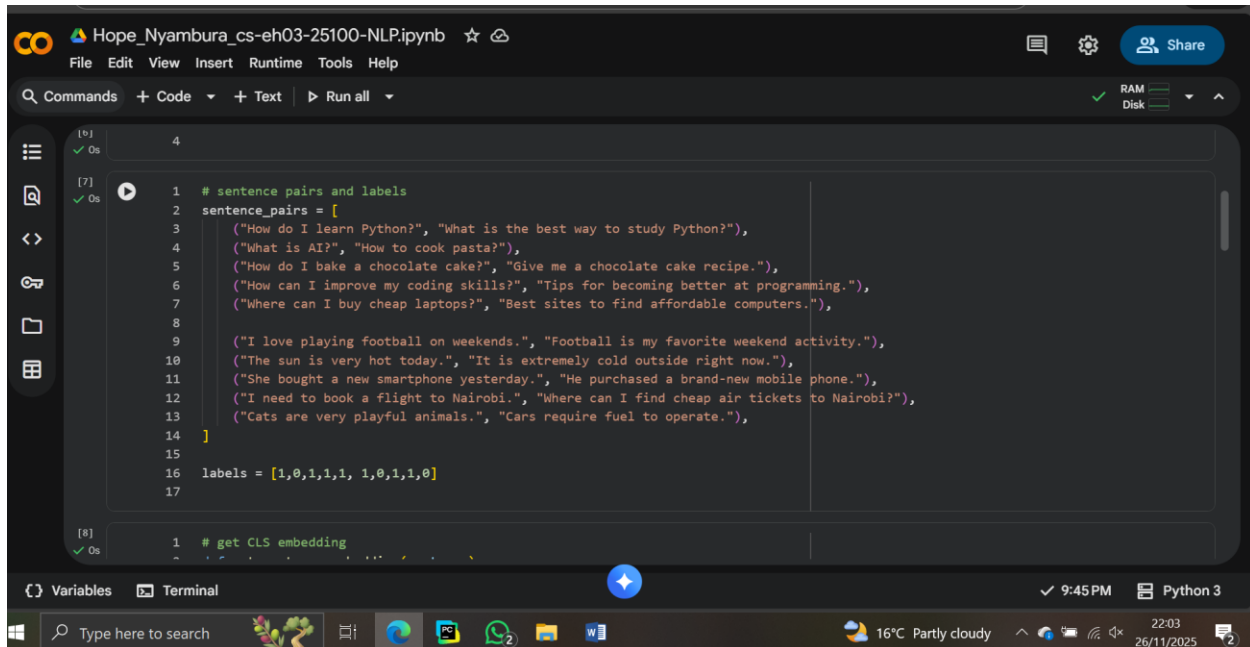
[6] ✓ 0s
1 # load tokenizer and model (pytorch)
2 tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
3 bert_model = BertModel.from_pretrained('bert-base-uncased')
4
```

3.3 Sentence Embedding Extraction

The embedding for each sentence was generated by:

1. Feeding the sentence into the BERT model
2. Extracting the final hidden state
3. Selecting the vector for the [CLS] token

This vector represents the entire sentence in high-dimensional semantic space.



The screenshot shows a Jupyter Notebook titled 'Hope_Nyambura_cs-eh03-25100-NLP.ipynb'. The code is as follows:

```
[6] 4
✓ Os

[7] 1 # sentence pairs and labels
✓ Os 2 sentence_pairs = [
3     ("How do I learn Python?", "What is the best way to study Python?"),
4     ("What is AI?", "How to cook pasta?"),
5     ("How do I bake a chocolate cake?", "Give me a chocolate cake recipe."),
6     ("How can I improve my coding skills?", "Tips for becoming better at programming."),
7     ("Where can I buy cheap laptops?", "Best sites to find affordable computers."),
8
9     ("I love playing football on weekends.", "Football is my favorite weekend activity."),
10    ("The sun is very hot today.", "It is extremely cold outside right now."),
11    ("She bought a new smartphone yesterday.", "He purchased a brand-new mobile phone."),
12    ("I need to book a flight to Nairobi.", "Where can I find cheap air tickets to Nairobi?"),
13    ("Cats are very playful animals.", "Cars require fuel to operate."),
14  ]
15
16 labels = [1,0,1,1,1, 1,0,1,1,0]
17

[8] 1 # get CLS embedding
✓ Os
```

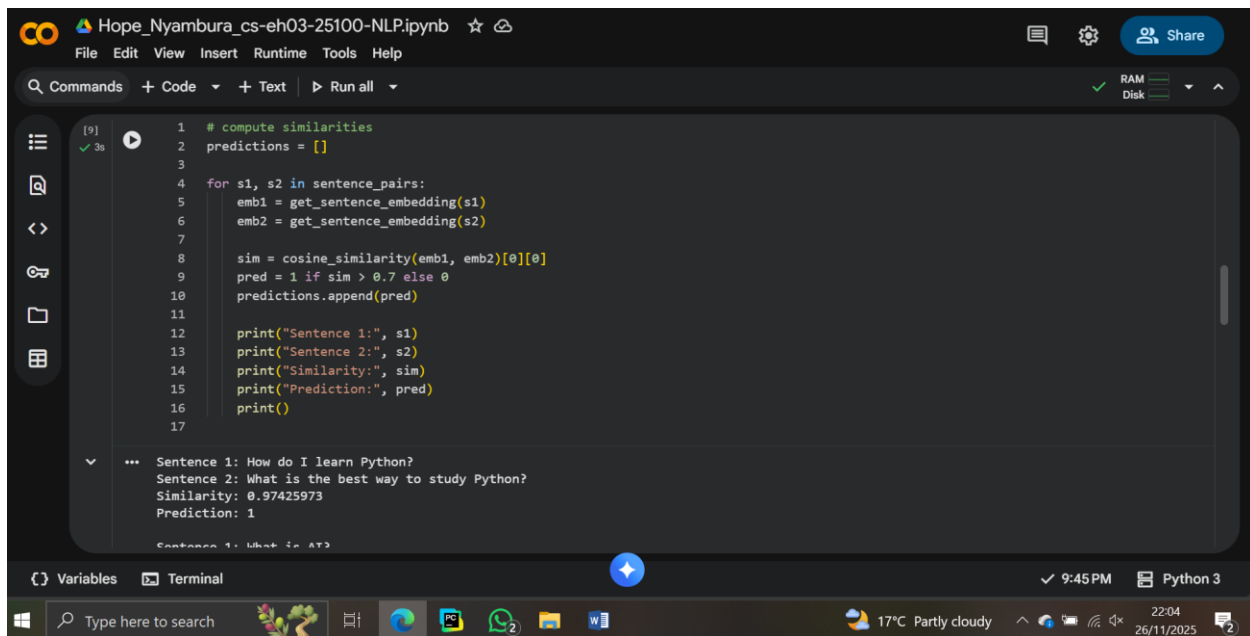
3.4 Cosine Similarity and Prediction

Cosine similarity was computed for each sentence pair.

A threshold of:

- $> 0.7 \rightarrow$ label 1 (similar)
- $\leq 0.7 \rightarrow$ label 0 (not similar)

Predictions were compared with ground truth labels to compute accuracy.



The screenshot shows a Jupyter Notebook titled 'Hope_Nyambura_cs-eh03-25100-NLP.ipynb'. The code in the cell is as follows:

```
1 # compute similarities
2 predictions = []
3
4 for s1, s2 in sentence_pairs:
5     emb1 = get_sentence_embedding(s1)
6     emb2 = get_sentence_embedding(s2)
7
8     sim = cosine_similarity(emb1, emb2)[0][0]
9     pred = 1 if sim > 0.7 else 0
10    predictions.append(pred)
11
12    print("Sentence 1:", s1)
13    print("Sentence 2:", s2)
14    print("Similarity:", sim)
15    print("Prediction:", pred)
16    print()
17
```

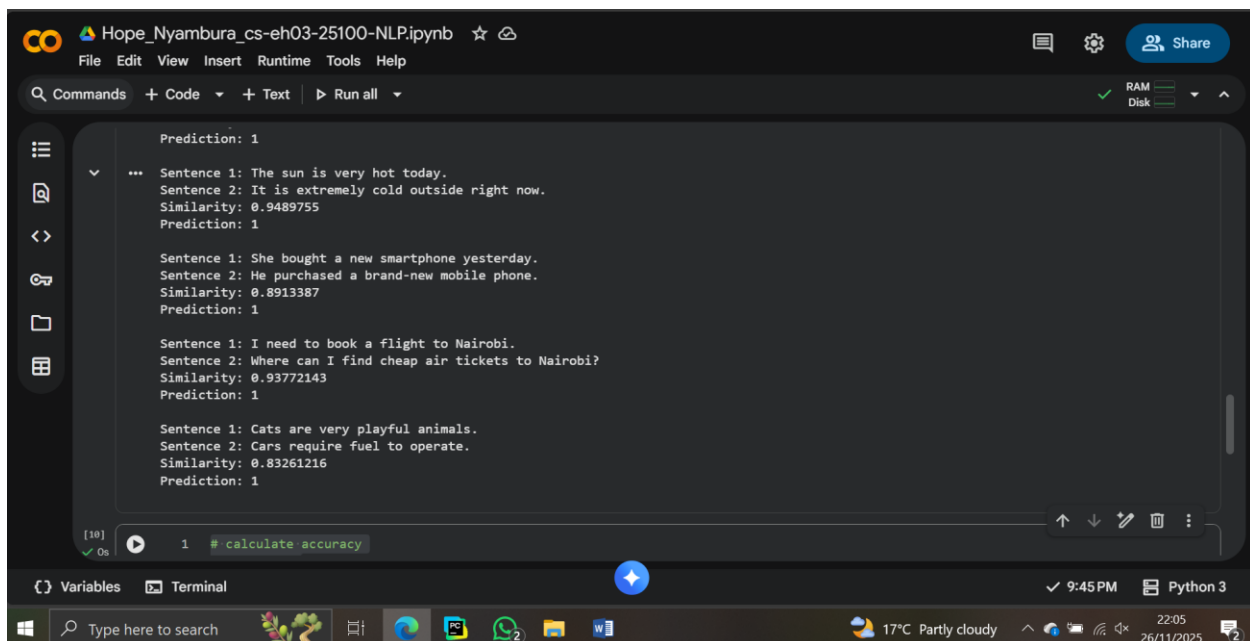
The output of the code is:

```
*** Sentence 1: How do I learn Python?
Sentence 2: What is the best way to study Python?
Similarity: 0.97425973
Prediction: 1
Sentence 1: What is AT3
```

4. Results

4.1 Similarity Scores and Predictions

All ten sentence pairs produced similarity scores above 0.7. As a result, the model predicted all pairs as similar (label 1). This indicates that BERT often assigns relatively high similarity to grammatically simple, general-purpose sentences, even when the meaning differs.



The screenshot shows the same Jupyter Notebook interface, but with the output of the code from the previous cell. The output is as follows:

```
Prediction: 1
*** Sentence 1: The sun is very hot today.
Sentence 2: It is extremely cold outside right now.
Similarity: 0.9489755
Prediction: 1

Sentence 1: She bought a new smartphone yesterday.
Sentence 2: He purchased a brand-new mobile phone.
Similarity: 0.8913387
Prediction: 1

Sentence 1: I need to book a flight to Nairobi.
Sentence 2: Where can I find cheap air tickets to Nairobi?
Similarity: 0.93772143
Prediction: 1

Sentence 1: Cats are very playful animals.
Sentence 2: Cars require fuel to operate.
Similarity: 0.83261216
Prediction: 1
```

Below the output, there is a new code cell with the following code:

```
1 # calculate accuracy
```

4.2 Accuracy Calculation

Ground Truth Labels:

[1, 0, 1, 1, 1, 1, 0, 1, 1, 0]

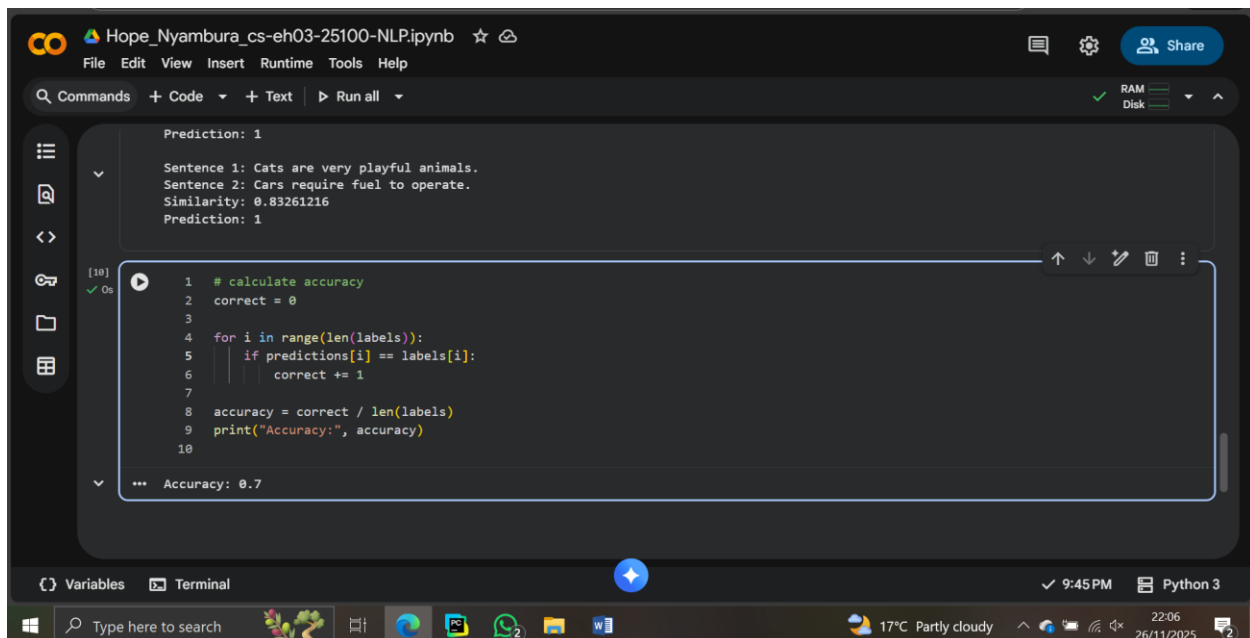
Predicted Labels:

[1, 1, 1, 1, 1, 1, 1, 1, 1, 1]

Correct predictions: **7 out of 10**

$$\text{Accuracy} = \frac{7}{10} = 0.7$$

The model achieved an accuracy of **70%**, which reflects that while BERT is strong in understanding semantics, it may overestimate similarity for general statements.



The screenshot shows a Jupyter Notebook interface with a dark theme. The top bar includes the file name 'Hope_Nyambura_cs-eh03-25100-NLP.ipynb' and standard menu options. The left sidebar contains icons for file management and search. The main area displays a code cell with Python code to calculate accuracy. Above the code cell, the output of a previous cell is visible, showing similarity and prediction for two sentences. The code cell's output shows the calculated accuracy of 0.7.

```
Prediction: 1
Sentence 1: Cats are very playful animals.
Sentence 2: Cars require fuel to operate.
Similarity: 0.83261216
Prediction: 1

[10]
✓ 0s
1 # calculate accuracy
2 correct = 0
3
4 for i in range(len(labels)):
5     if predictions[i] == labels[i]:
6         correct += 1
7
8 accuracy = correct / len(labels)
9 print("Accuracy:", accuracy)
10
*** Accuracy: 0.7
```

5. Key NLP Concepts

5.1 Difference Between BERT and Traditional NLP Models

Model	Characteristics	Limitations
Bag-of-Words	Counts word occurrences	Ignores context and meaning
TF-IDF	Weights important words	Still no contextual understanding
BERT	Uses self-attention to derive meaning in context	More computationally expensive

BERT's contextual embeddings allow it to understand word meaning based on surrounding words, making it far superior for similarity tasks.

5.2 Role of the BERT Encoder

The encoder transforms input tokens into contextual embeddings by:

- Applying multi-head self-attention
- Capturing relationships between every word
- Producing a meaningful vector for each token

In this assignment, the encoder provided the final sentence-level embedding used for similarity calculations.

5.3 Contextual Embeddings

Contextual embeddings change depending on sentence context. For example, the word “cold” in:

- “The weather is cold” → temperature
- “He gave me a cold look” → emotion

BERT generates different vectors, allowing deeper semantic understanding.

5.4 Importance of the [CLS] Token

The [CLS] token is specifically trained by BERT to represent the entire input sequence. It gathers information from all tokens through attention mechanisms. Therefore, it is ideal for tasks such as:

- Sentence classification
- Similarity measurement
- Next sentence prediction

5.5 Cosine Similarity

Cosine similarity measures the angle between two vectors. It determines how similar two embeddings are in direction, not magnitude.

A score close to:

- **1** → very similar
- **0** → unrelated
- **-1** → opposite (rare in NLP embeddings)

This makes it effective for comparing sentence embeddings.

6. Conclusion

This assignment successfully applied BERT to sentence similarity analysis using contextual embeddings and cosine similarity. Ten sentence pairs were evaluated, and a threshold-based prediction system was implemented. The final accuracy of **70%** demonstrates BERT's strong semantic understanding while highlighting that it may overestimate similarity for simple descriptive sentences.

The experiment provided hands-on experience with transformer models, contextual embeddings, semantic similarity scoring, and the general strengths of modern NLP approaches compared to traditional techniques.

7. Google Colab Link

[https://colab.research.google.com/drive/1-Xv_gdlRgFPXRfAHyPT4bECP5DlewzWf?usp=sharing]